ERHVERVSAKADEMI KØBENHAVN

Database Course

Session 1: Onboarding & Calibration

Welcome • Self-assess • Tools • Theory • Hands-on

What we should cover today

- We know where the class is collectively (self-assessment + discussion)
- Everyone has the tooling set up (or a working fallback plan)
- We cover the minimum theory needed to start writing SELECT queries
- We run a short guided SQL exercise together

Principle for today: keep it practical. If installs take longer, we reduce theory and focus on “SQL runs”.

Self-assessment

Scale:
- 0 = never
- 1 = recognise
- 2 = can do with help
- 3 = can do independently

Rate yourself on:
- Tables / rows / columns
- Primary key
- SELECT, WHERE, ORDER BY
- INSERT / UPDATE / DELETE
- GROUP BY + aggregates
- JOINs

Format:
- 5 min self-exercise, then class discussion
- When you share: name + education + work background (mandatory)

Also tell me:
- Have you installed a database before?
- Have you used a GUI client (e.g., DBeaver)?
- What do you want to be able to do by the end of the course?

Tooling for this course

Goal: everyone can run SQL queries today

Required
- Docker installed
- MariaDB running locally in Docker (container: db1, volume: mariadb_data)
- DBeaver installed (GUI client)

Recommended
- A folder for the course (.sql files)
- A text editor for saving and reviewing SQL

Setup checklist

Step-by-step (important info only)

1) Pull image
   docker pull mariadb:latest

2) Create persistent volume
   docker volume create mariadb_data

3) Run database (choose your own password)
   docker run --name db1 -e MARIADB_ROOT_PASSWORD=<your_password> \
     -p 3306:3306 -v mariadb_data:/var/lib/mysql -d mariadb:latest

4) DBeaver connection fields
   Host: localhost
   Port: 3306
   User: root
   Password: <your_password>
   Database: (leave blank)

5) Quick sanity query
   SELECT 1;

Troubleshooting: connection issues

Fast checks (in order):

- Is the container running? (db1)
- Is port 3306 free on your machine?
- In DBeaver: Host must be localhost / 127.0.0.1
- Is the password correct? (watch keyboard layout issues)
- If still stuck: pair up and continue; we will resolve later

Theory: what SQL is

Just enough theory to start querying

SQL is a language for asking precise questions of relational data.

- Relational databases store data in tables (rows + columns)
- A query describes what you want (not how to compute it)
- We care about: correctness, readability, and repeatability
- Today we focus on reading and writing simple SELECT queries

Key vocabulary

- Table
- Row
- Column
- Schema
- Query
- Result set

Theory: keys and relationships

Primary key (PK)

Uniquely identifies a row in a table.

Foreign key (FK)

A column that references a PK in another table (relationship).

Why this matters:

- Prevents duplicate and inconsistent data
- Makes JOINs possible (link tables correctly)
- Supports reliable updates and deletions

Theory: the anatomy of a SELECT query

This is the core pattern you will use constantly

```
SELECT column1, column2
FROM   table_name
WHERE  condition
ORDER BY column1;
```

- FROM: choose the dataset
- WHERE: filter rows
- SELECT: choose columns
- ORDER BY: sort output
- LIMIT: cap rows (if supported)

Two practical rules:

- Write queries to match requirements (what should the result contain?)
- When debugging, simplify: start with SELECT * FROM, then add WHERE, then add SELECT columns

Guided hands-on

Tasks (run on today's demo table)

- SELECT * from the demo table
- Select only 2 columns (e.g., name + city)
- Filter rows with WHERE (e.g., city = …)
- Sort with ORDER BY (e.g., age descending)
- Count rows (COUNT(*))

If you finish early:

- Try IN / BETWEEN / LIKE (optional)
- Try filtering NULL values (IS NULL)
- Write one query from a plain-English requirement